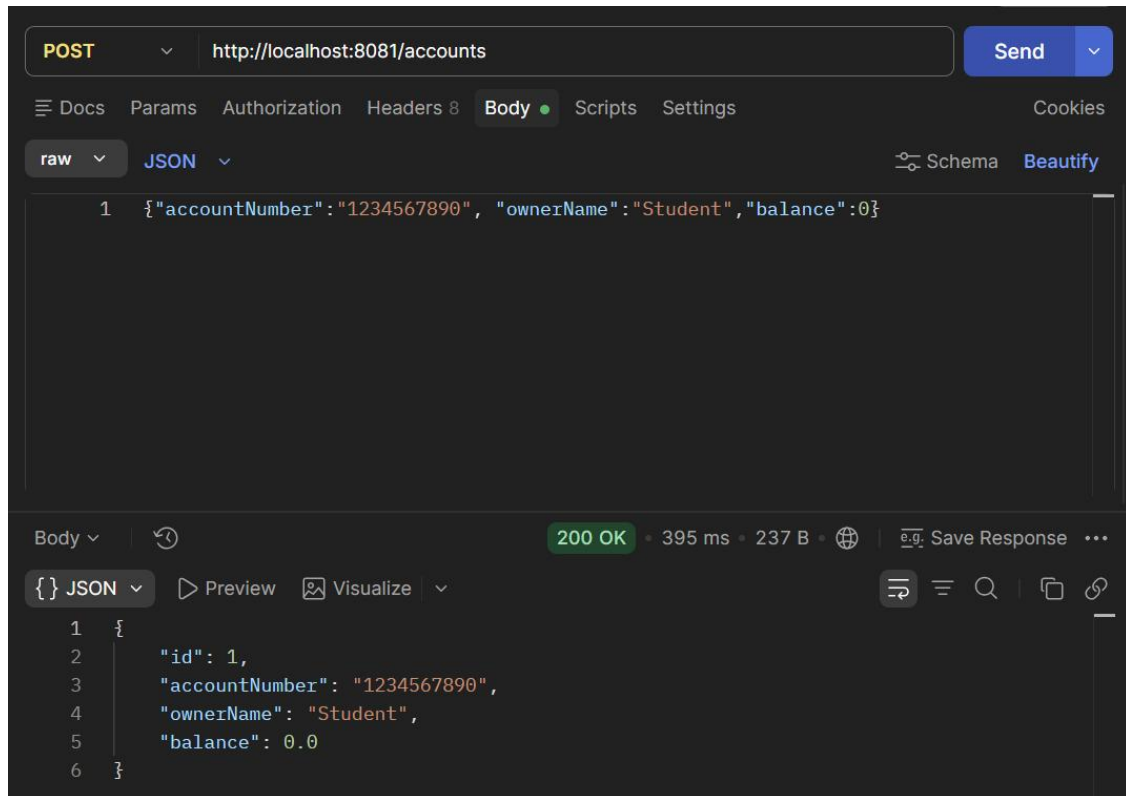


Lab9 Transaction

ชื่อ สรวิชญ์ วันแสน รหัสนักศึกษา 673380064-8 กลุ่มเรียน 1

การสร้างบัญชี



การฝากเงินสำเร็จ

The screenshot displays a REST client interface with the following details:

- Request:**
 - Method: **POST**
 - URL: `http://localhost:8081/accounts/1/deposit`
 - Body (JSON):

```
1 { "amount": 1000 }
```
- Response:**
 - Status: **200 OK**
 - Time: 159 ms
 - Size: 196 B
 - Body (JSON):

```
1 {  
2   "message": "Deposit successful"  
3 }
```

การตรวจสอบข้อมูลบัญชี

GET http://localhost:8080/accounts/{id}

The screenshot shows a REST client interface with the following details:

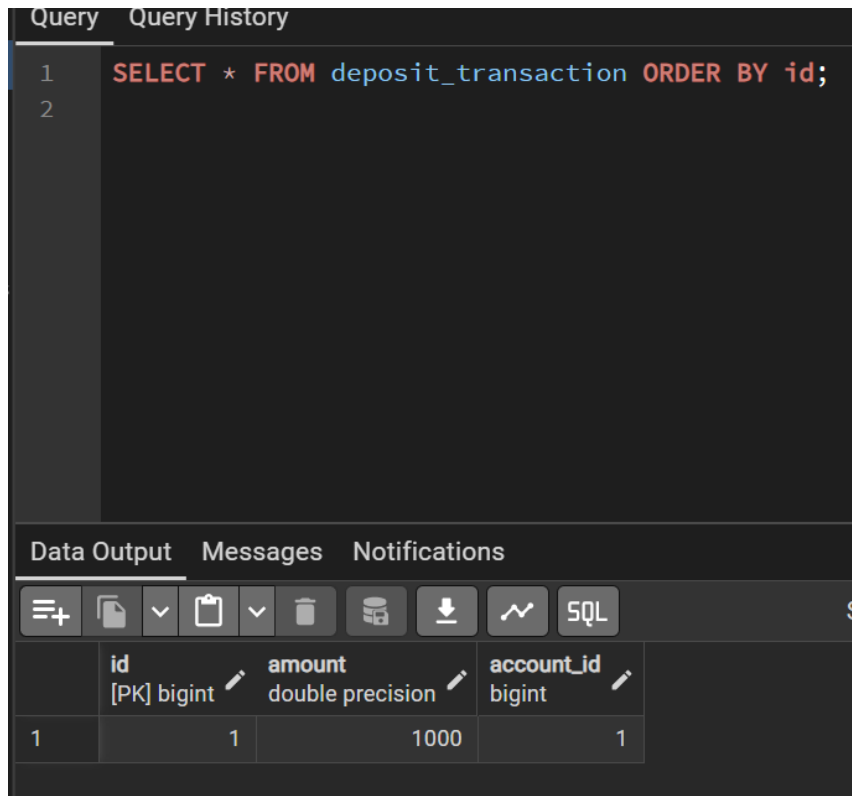
- Method:** GET
- URL:** http://localhost:8081/accounts/1
- Send Button:** A blue button labeled "Send".
- Tabs:** Docs, Params, Authorization, Headers 8, Body (selected), Scripts, Settings, Cookies.
- Query Params:** A table with columns Key, Value, Description, Bulk Edit, and
- Body:** A dropdown menu showing "JSON".
- Status:** 200 OK, 29 ms, 240 B.
- Response Body:** A JSON object:

```
{  "id": 1,  "accountNumber": "1234567890",  "ownerName": "Student",  "balance": 1000.0}
```

จากภาพสังเกตได้ว่า ยอด balance เปลี่ยนจาก 0 เป็น 1000 ส่วนเลขบัญชีและชื่อเจ้าของบัญชียังเหมือนเดิม แสดงว่าระบบเพิ่มยอดเงินให้บัญชีที่เลือกได้ถูกต้อง

4 การตรวจสอบประวัติฝากเงิน

SELECT * FROM deposit_transaction ORDER BY id;



จากภาพสังเกตได้ว่า พบประวัติ 1 รายการ โดย amount เป็น 1000 และ account_id ตรงกับ id ของบัญชีที่ฝากเงิน เลข id ของประวัติอาจต่างกันได้ตามจำนวนครั้งที่เคยทดลอง

Account หนึ่งบัญชีมี DepositTransaction ได้หลายรายการ จึงเป็นความสัมพันธ์ One-to-Many เมื่อมองจากบัญชี ส่วนในโค้ดกำหนด @ManyToOne ที่ DepositTransaction เพื่ออ้างอิงกลับไปยังบัญชี

การทำงานของโค้ด

Controller รับค่าขอจาก Postman แล้วเรียก Service ส่วน Repository ใช้อ่านและบันทึกข้อมูล เมธอด deposit ใน DepositService มี @Transactional ครอบไว้ จึงรวมการเปลี่ยนยอดเงินกับการสร้างประวัติไว้ใน Transaction เดียว

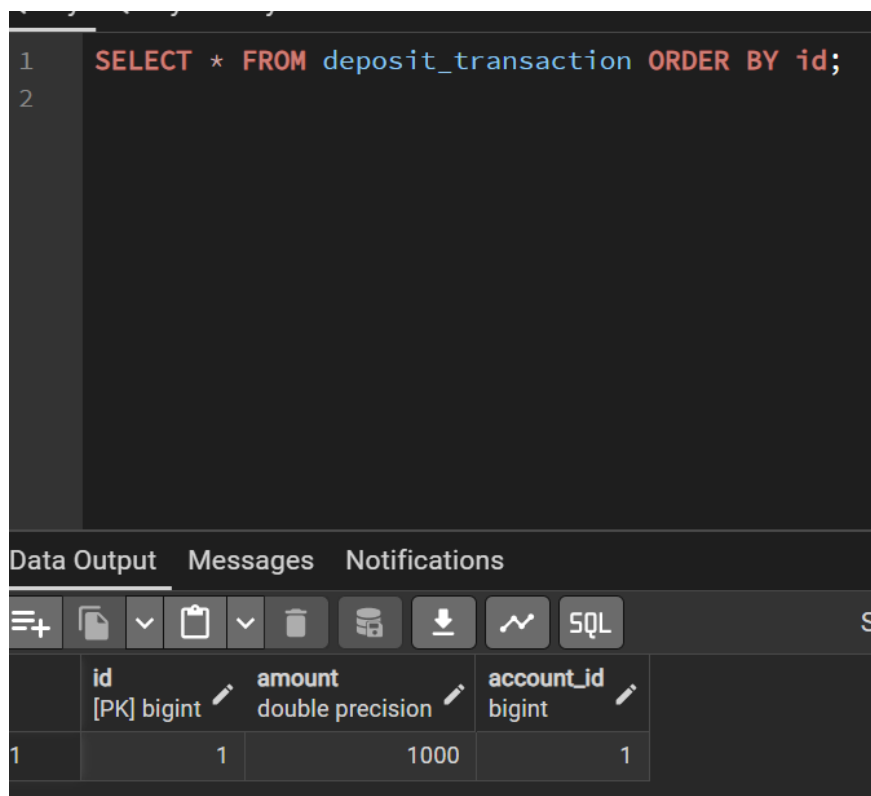
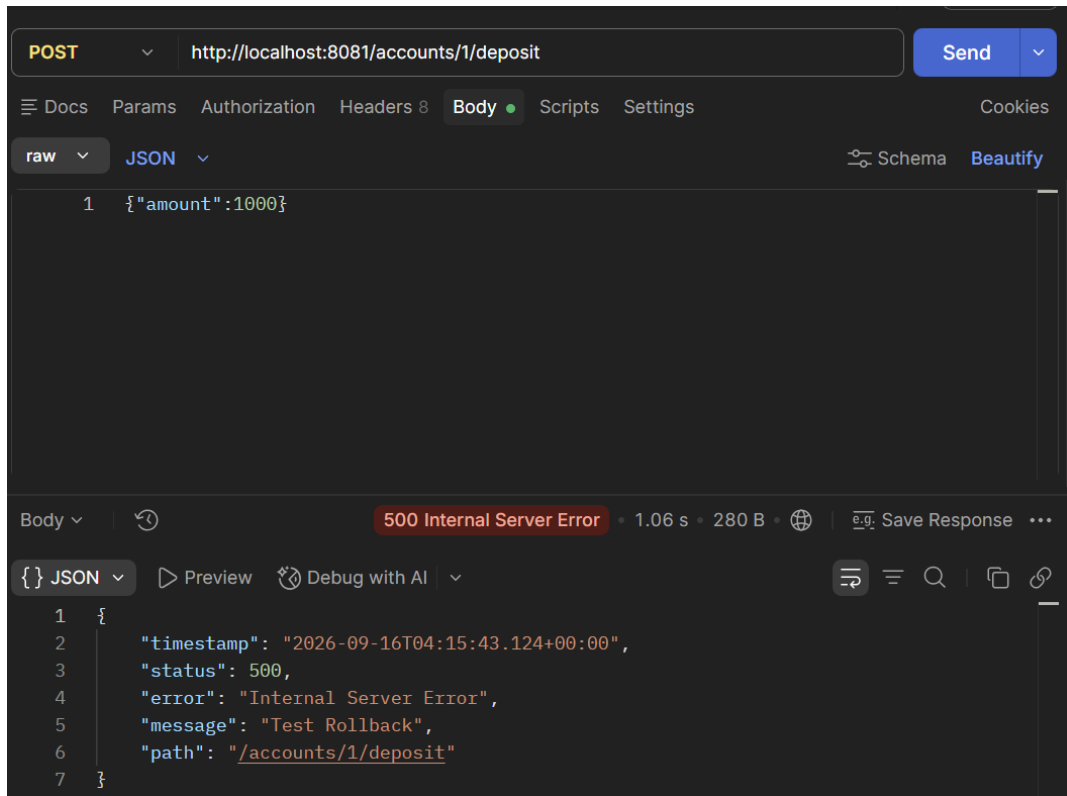
โค้ดใช้ saveAndFlush เพื่อให้ส่ง SQL ก่อนจบเมธอด แต่การส่ง SQL ยังไม่ใช่ COMMIT ถ้าเกิด RuntimeException ภายหลัง ข้อมูลที่ส่งไปแล้วก็ยังถูก ROLLBACK ได้

5 การทดลอง Rollback

เริ่มจากยอด 1000 และประวัติ 1 รายการ เปิดบรรทัดต่อไปนี้ท้ายเมธอด deposit หลังบันทึกทั้งสองตารางแล้ว จากนั้นหยุดโปรแกรมและรันใหม่

```
throw new RuntimeException("Test Rollback");
```

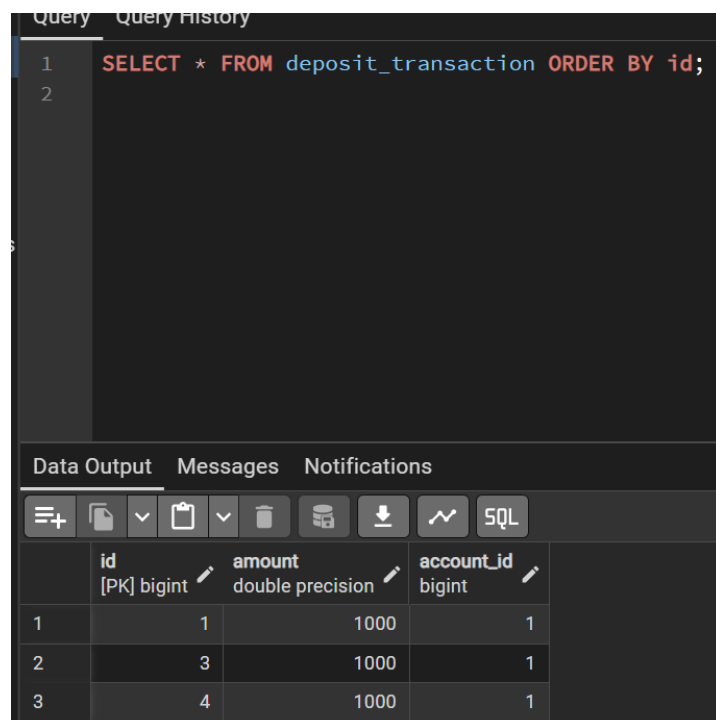
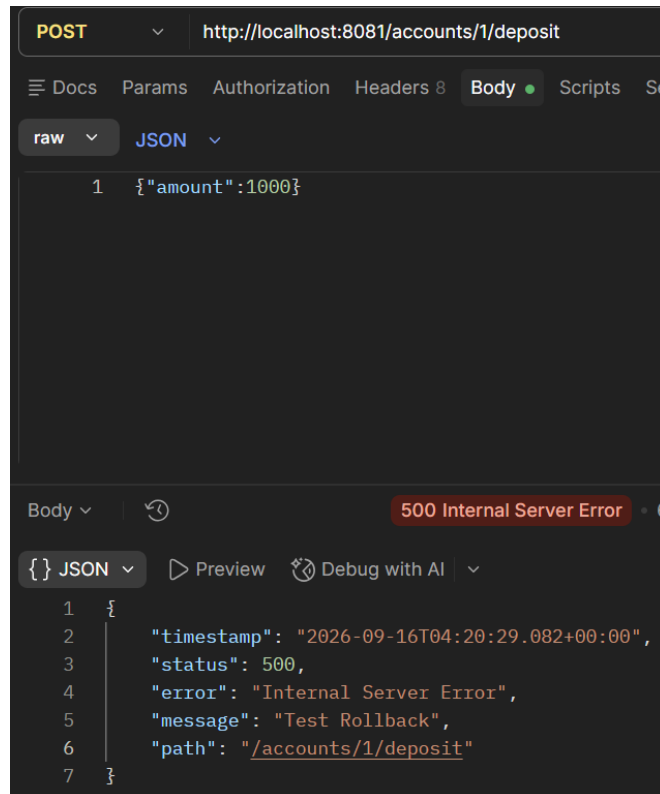
ส่งค่าขอฝากเงินเพิ่ม 1000 อีกครั้ง โดยยังใส่ @Transactional ไว้



6 เปรียบเทียบเมื่อเอา Transaction ออก

เอา @Transactional ออกจาก deposit แต่ยังเปิดบรรทัด throw ไว้ หยุดโปรแกรมแล้วรันใหม่ จากนั้นฝากเพิ่ม 1000 อีกครั้ง

[แทรกภาพ ผล HTTP 500 ยอดบัญชี และตารางประวัติเมื่อไม่มี Transaction]



กรณีทดสอบ	ยอดหลัง error	ประวัติหลัง error
มี @Transactional	1000	1 รายการ
ไม่มี @Transactional	2000	2 รายการ

ตารางเป็นผลที่คาดหวังเมื่อทดลองตามลำดับนี้ แม้ทั้งสองกรณีตอบ 500 เหมือนกัน แต่เมื่อไม่มี Transaction ที่ Service คำสั่งบันทึกของ Repository จะ commit แยกกัน
ข้อผิดพลาดท้ายเมธอดจึงไม่ย้อนข้อมูลที่บันทึกไปแล้ว

หลังทดลอง ใส่ @Transactional กลับและ comment บรรทัด throw เพื่อให้โปรแกรมฝากเงินได้ตามปกติ

1. หน้าที่ของ Transaction

ตอบ ทำให้หลายคำสั่งเป็นงานเดียวกัน ถ้าสำเร็จก็บันทึกทั้งหมด แต่ถ้าเกิดข้อผิดพลาดอย่างในแลบ ก็ยกเลิกทั้งหมด

2. เหตุผลที่การฝากเงินควรใช้ Transaction

ตอบ เพื่อให้ยอดเงินและประวัติฝากเงินถูกบันทึกพร้อมกัน ถ้าส่วนใดล้มเหลว ต้องยกเลิกทั้งคู่เพื่อให้ข้อมูลตรงกัน

3. ความต่างระหว่าง COMMIT และ ROLLBACK

ตอบ COMMIT คือยืนยันการบันทึกข้อมูล ส่วน ROLLBACK คือยกเลิกการเปลี่ยนแปลงแล้วกลับไปใช้ข้อมูลเดิม

4. ความสัมพันธ์ระหว่างสองตาราง

ตอบ เป็นแบบ One-to-Many คือหนึ่งบัญชีมีประวัติฝากเงินได้หลายรายการ แต่แต่ละรายการเป็นของบัญชีเดียว

5. ข้อมูลเมื่อเกิดข้อผิดพลาดขณะใช้ Transaction

ตอบ ในแลบนี้ ยอดเงินและประวัติจะกลับไปเหมือนก่อนฝากครั้งที่เกิดข้อผิดพลาด ส่วนข้อมูลเก่ายังอยู่

6. ผลเมื่อเอา Transaction ออก

ตอบ แต่ละคำสั่งบันทึกข้อมูลแยกกัน ถึงเกิดข้อผิดพลาดตอนท้าย ยอดเงินและประวัติที่บันทึกไปแล้ว ก็ยังอยู่